

Assignment 2

Question 1: Self-Assessment (LLM, Deep Learning, AI and ML)

Name: Ankit Kumar Singh

I rated myself on the A / B / C scale below. For each area I have given the reasoning and the work that backs it up, so the rating can be checked.

Summary

Area	Rating	Proof (summary)
Machine Learning (ML)	A	DDoS Attack Detection System (CIC-DDoS2019, 4M+ records): 14% accuracy gain over baseline models; PCA cut features by 40% and computation time by 25%. Also the Python for AI and Data Science certificate.
Deep Learning (DL)	B	Deepfake Detection System (images and video) built with PyTorch: CNN, ResNeXt-50, LSTM and Vision Transformer combined into an ensemble, with a Flask web app and Docker deployment. Also built DL models with TensorFlow and Keras in the certificate.
Artificial Intelligence (AI)	B	Applied ML and deep learning to real problems (DDoS detection, deepfake detection) and built a working RAG assistant; understand how AI systems are put together (see Assignment 2 answers).
Large Language Models (LLM)	B	RAG Document Q&A Assistant: PDF upload, chunking, local Hugging Face embeddings, ChromaDB vector search, and a local Llama 3.2 3B model served through Ollama, with a FastAPI backend and React frontend.

Machine Learning: Rating A

Project: DDoS Attack Detection System

I built a machine learning system that detects DDoS attacks in network traffic, using the CIC-DDoS2019 dataset (more than 4 million records). I compared my models against baseline ML models and improved accuracy by 14%. To keep training practical on such a large dataset, I applied PCA, which reduced the feature set by 40% and cut computation time by 25%.

Proof: [DDoS Attack Detection notebook \(GitHub\)](#)

Certificate: Python for AI and Data Science (Jun 2024 to Jul 2024)

- Built and trained ML models in Python using Scikit-learn on datasets with over 1 million records.
- Applied data preprocessing, feature engineering, and hyperparameter tuning to improve model performance.
- Automated preprocessing workflows, reducing manual data preparation time by 40%.

Why an A

I handled the full workflow on my own: cleaning a large dataset, reducing features, training and comparing models, and measuring the improvement.

Deep Learning: Rating B

Project: Deepfake Detection System (Images and Video)

I built a deep learning system that detects deepfakes in both images and videos. Instead of relying on one network, it combines four different models and fuses their outputs:

- **CNN:** 4 convolutional blocks with batch normalization for spatial features.
- **ResNeXt-50:** pre-trained on ImageNet and fine-tuned through transfer learning.
- **LSTM:** a ResNet50 feature extractor with a bidirectional LSTM to catch inconsistencies across video frames.
- **Vision Transformer:** self-attention to model global relationships in the image.
- **Ensemble:** a gating network that adaptively weights the four models.

Around the models I built the preprocessing pipeline (resizing to 224x224, ImageNet normalization, augmentation, and selecting 16 key frames per video), training and inference scripts, a Flask web app with REST endpoints for image and video detection, and Docker files for deployment. The project is written in PyTorch. The README reports roughly 95% accuracy and 0.98 ROC-AUC on the test dataset.

Proof: [Deepfake Detection repository \(GitHub\)](#)

Why a B: I can design and train networks and put them behind an application. I am still building depth in areas like model interpretability and real-time processing, which are on my improvement list for this project.

Artificial Intelligence: Rating B

I have applied machine learning and deep learning to two real problems, DDoS detection on network traffic and deepfake detection on images and video, and I have built a working retrieval-augmented question-answering assistant. For this assignment I also researched how AI applications are designed in practice, including chatbot architecture and vector databases (see my answers to Questions 2 and 3).

Proof: [DDoS Attack Detection](#) | [Deepfake Detection](#) | [RAG Document Assistant](#)

Why a B: my hands-on work covers ML, deep learning, and LLM applications, and I am still broadening into areas like agents and larger production systems.

Large Language Models: Rating B

Project: RAG Document Q&A Assistant

I built an application where a user uploads a PDF and asks questions about it. It uses Retrieval-Augmented Generation (RAG): the system finds the most relevant parts of the document and gives them to a language model, so answers come from the document rather than from the model's memory. The pipeline works like this:

- The React frontend sends the uploaded PDF and the user's question to a FastAPI backend.
- The backend extracts the PDF text and splits it into chunks.
- Each chunk is converted to an embedding using a local Hugging Face model and stored in ChromaDB, a vector database.
- For each question, a similarity search retrieves the most relevant chunks.
- Those chunks are passed to Llama 3.2 3B, running locally through Ollama, which generates the answer.
- The answer is shown in the UI along with source page references, so the user can check where it came from.

Because the embeddings and the language model both run locally, the project needs no paid API credits.

Proof: [RAG Document Assistant repository \(GitHub\)](#)

Why a B

I have built a complete RAG application on my own, from document processing to vector search to answer generation. To reach an A, I want to add a proper evaluation set to measure answer quality, try hybrid search and re-ranking to improve retrieval, and add stronger guardrails.

Note on the ratings

I kept these ratings honest and tied to work I can show and explain. I am happy to walk through any of the projects above in detail.

Question 2

Key architectural components of an LLM-based chatbot

The big picture

The first thing I learned while building my RAG assistant is that the language model is only one piece of a chatbot. The model takes text in and gives text out. It does not know your documents, it does not remember the last conversation, and it cannot click or call anything by itself. Everything else in the system exists to fix those gaps: deciding what text the model sees, what it is allowed to do, and whether the answer is safe to show.

Figure 1 shows how I picture it. The backend (orchestrator) sits in the middle and talks to every other part. The user never talks to the model directly.

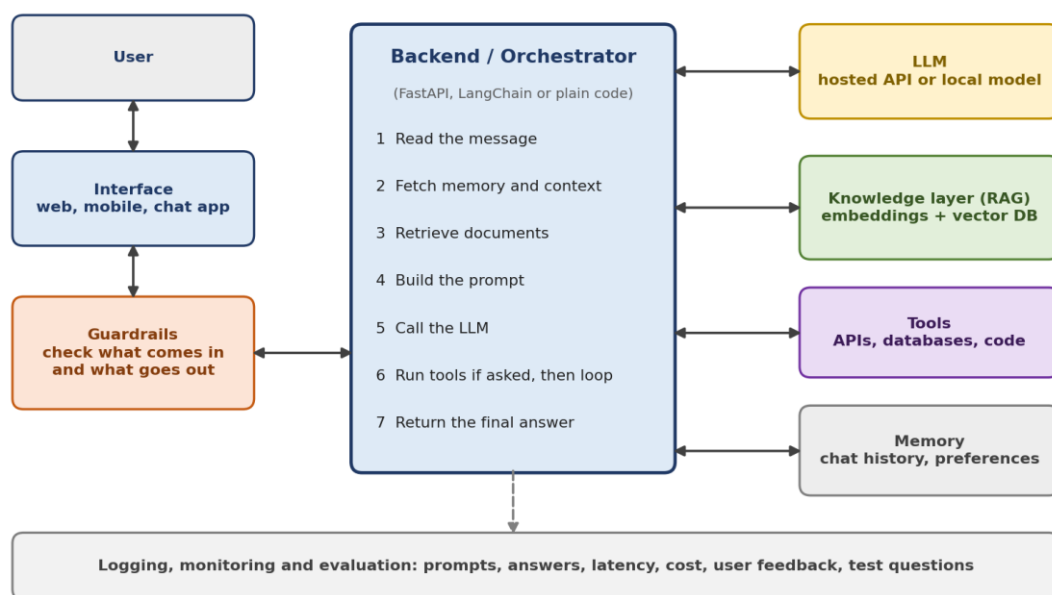


Figure 1. Main components of an LLM-based chatbot. Arrows show which parts talk to each other.

What each component does

Interface. This is what the user sees: a web page, a mobile app, Slack, or WhatsApp. Its job is simple, send the message and show the reply. One detail that matters a lot is streaming. If the answer appears word by word, the bot feels fast even when the model needs several seconds.

Backend (orchestrator). This is the coordinator, and in my project it is a FastAPI service. For every message it reads the input, gathers memory and documents, builds the prompt, calls the model, runs any tools the model asks for, and returns the answer. Frameworks like LangChain or LlamaIndex can do this work, but plain code is fine for a small app and easier to understand.

The LLM. The choice is usually between a hosted model (Claude, GPT, Gemini) and one you run yourself (Llama, Mistral). Hosted models are stronger and easy to start with, but they cost money per request and your data leaves your machine. A local model is private and free to run, but smaller ones reason less well. I picked Llama 3.2 3B through Ollama for my project because I wanted no API costs, and I accepted the weaker answers as the price.

Prompt and context management. The model only knows what is in the text we send it on each turn, so this text has to be built carefully. It usually has three parts: a system prompt (the role, tone, and rules), the recent conversation, and any extra information such as retrieved document chunks. The context window is limited, so long conversations are trimmed or summarized. Whatever does not fit is simply invisible to the model.

Knowledge layer (RAG). A model does not know your private files, and retraining it for every new document is not practical. Retrieval-Augmented Generation solves this by looking up the relevant pieces at question time and handing them to the model. I explain this in detail in the next section.

Tools. Through function calling, the model can reply with a request such as "look up order 4821" instead of a normal answer. The backend runs that action, for example a database query or an API call, and gives the result back to the model, which then writes the final reply. This is the step that turns a chatbot into something that can actually do things.

Memory. Short-term memory is the current conversation, which is reset with each request. Long-term memory, like a user's preferences or past issues, has to be saved in a database and pulled back in only when it is relevant, otherwise the prompt fills up with noise.

Guardrails. These are checks on both sides of the model. On the way in, they catch prompt injection (text that tries to override the rules) and off-topic or harmful requests. On the way out, they check for unsafe content and leaked personal data. For risky actions, like deleting records or sending money, a human should confirm first.

Logging and evaluation. Every prompt, answer, delay, and cost should be logged, along with user feedback. A set of test questions with expected answers lets you check whether a change to the prompt, model, or retrieval made things better or worse. I think this is the part beginners skip most often, and I want to add it to my own project next.

How the knowledge layer (RAG) works

RAG has two separate phases, and it helps to keep them apart in your head. Phase A happens ahead of time, when documents are prepared. Phase B happens live, every time someone asks a question.

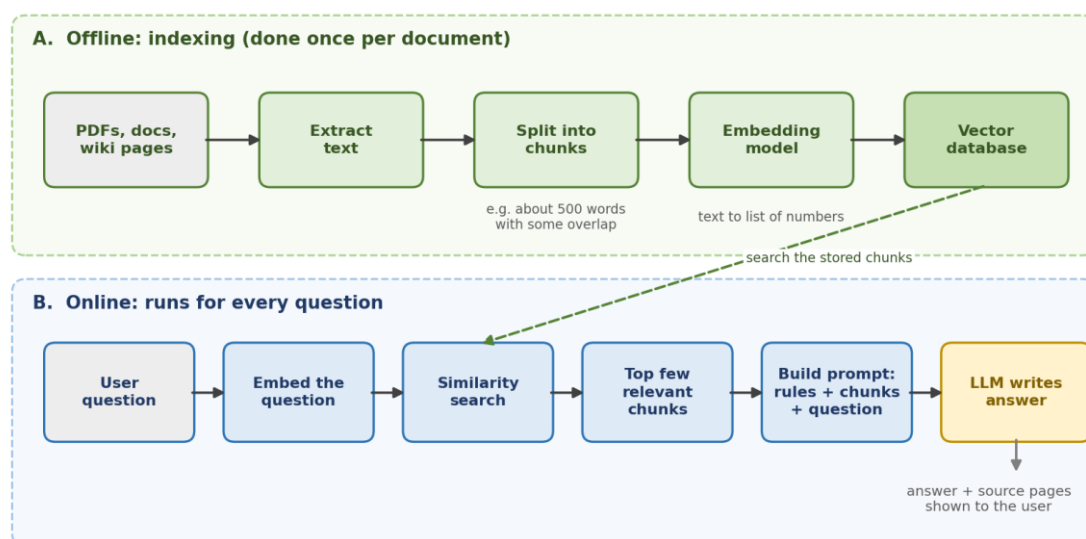


Figure 2. RAG workflow. The vector database connects the offline preparation with the live question answering.

Phase A, indexing. The documents are read, the text is extracted, and it is split into chunks. Chunk size is a real decision. Very large chunks bring in a lot of unrelated text, and very small chunks lose context, so a moderate size with a small overlap between neighbors is a common starting point. Each chunk then goes through an embedding model, which turns it into a list of numbers that represents its meaning, and the result is saved in a vector database.

Phase B, answering. The user's question is embedded with the same model, and the database returns the chunks whose numbers are closest to it. Those chunks are placed in the prompt together with the rules and the question, and the LLM writes an answer using that material. Because we know which chunks were used, the app can also show source page numbers, which my assistant does.

A worked example, step by step

Say a user has uploaded a company policy PDF and asks: "How many days do I have to return an item?"

1. The interface sends the question to the backend, and the answer will stream back to the same screen.
2. The input guardrail checks the message for harmful content or injection attempts. It looks fine.
3. The backend loads the recent conversation and any saved preferences for this user.
4. The question is embedded, and the vector database returns the three or four chunks closest in meaning, for example the returns policy section.
5. The backend builds the prompt: the system rules (answer only from the provided text, say when unsure), the retrieved chunks, the conversation, and the question.
6. The LLM reads this and writes the answer. If it needed live data, such as an order status, it would ask for a tool call, the backend would run it, and the model would continue.
7. The output guardrail checks the answer, the source pages are attached, and the text is streamed to the user.
8. The exchange is logged with its latency and cost, and the conversation is saved for the next turn.

What usually goes wrong, and what helps

Problem	Why it happens	What helps
Made-up answers (hallucination)	The model fills gaps with things that sound right	Ground it with RAG, tell it to say "I don't know" when the context has no answer, and show sources
Wrong chunks retrieved	Chunks too big or small, or the question is worded differently	Tune chunk size and overlap, retrieve a few more chunks, add keyword search or re-ranking
Slow replies	Long prompts, big model, many steps	Stream the answer, use a smaller model for easy questions, cache repeated questions

Problem	Why it happens	What helps
Prompt injection	Text inside a document or message tells the model to ignore its rules	Treat retrieved text as data, filter inputs, limit what tools can do, confirm risky actions
Costs creeping up	Every turn resends the history and the retrieved text	Trim or summarize history, cap retrieved chunks, track tokens per request

How I would approach building one

I would start as simply as possible: a good model with a clear system prompt, and see how far that goes. If the bot needs to answer from private or changing documents, I would add RAG. If it needs to take actions, I would add tools. Fine-tuning comes last, only when prompting cannot give the consistent style or behavior I need. Along the way I would collect real questions and use them as a test set, so every decision is based on results and not on guesses.

Question 3

Vector databases, and which one I would choose

What a vector database is

A normal database is good at exact questions, like finding every row where the email equals a certain value. A vector database answers a different question: which stored items are most similar in meaning to this one? To make that possible, every piece of content (a paragraph, an image, a product) is first turned into an embedding, which is a long list of numbers produced by a model. Items with similar meaning end up with similar numbers, so they sit close together. The database stores these vectors and can quickly find the nearest ones to a query.

For example, "I forgot my login" and "How do I reset my password?" share almost no words, but their embeddings are close, so a vector search finds the password article anyway. That is what makes RAG work. I used this idea in my RAG assistant, where ChromaDB stores the PDF chunks and returns the ones closest to the user's question.

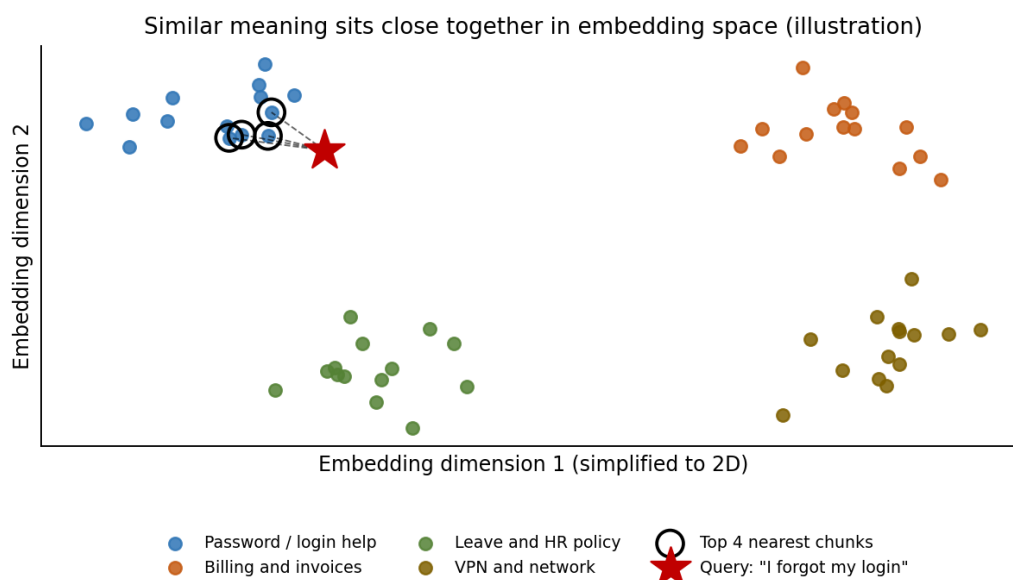


Figure 3. A query lands near the chunks with similar meaning. The real space has hundreds of dimensions; this is a 2D sketch.

How they work

Similarity. Closeness is measured with cosine similarity, dot product, or Euclidean distance. The right one depends on how the embedding model was trained.

Approximate search indexes. Comparing a query to every single vector is too slow once you have millions. So databases use approximate nearest neighbor indexes, which give up a little accuracy for a big gain in speed. HNSW is a layered graph that you walk from coarse to fine. It is fast and accurate but uses a lot of memory. IVF groups vectors into clusters and only searches the closest clusters, which saves memory but needs more tuning. Compression such as quantization shrinks the vectors further. The constant trade-off is between accuracy, speed, and cost.

Metadata filtering. In real apps you rarely want the closest chunks overall. You want the closest chunks this user is allowed to see, or from documents updated this year. How well a database combines filters with vector search differs a lot between products.

Hybrid search. Pure semantic search can miss exact things like error codes or product names. Many systems combine it with keyword search and merge the two rankings, which usually improves results.

A real example: the vector database in my RAG assistant

My RAG Document Q&A Assistant is a good small-scale example of everything above, because the vector database is the part that lets the app answer questions about a PDF it has never seen before. This is how it is used:

9. When a PDF is uploaded, the FastAPI backend extracts the text and splits it into chunks.
10. Each chunk is turned into an embedding by a local Hugging Face model.
11. The chunks and their embeddings are stored in ChromaDB, along with enough information to know which page each chunk came from.
12. When the user asks a question, the question is embedded in the same way, and ChromaDB runs a similarity search to find the closest chunks.
13. Those chunks are passed to Llama 3.2 3B through Ollama to write the answer, and the page information lets the UI show source references.

Repository: [RAG Document Assistant \(GitHub\)](#)

Why ChromaDB fit this project. It is lightweight and free, and it works as part of the application without a separate database server to install and manage. That matched the goals of the project: everything runs locally, including the embeddings and the language model, with no paid API credits.

What this example taught me about choosing. In my project there is one user, a small number of documents, and no question of who is allowed to see what, so a simple embedded database is exactly right. The company scenario below is different: many users, permissions on every document, exact-term searches, and a lot more data. Those needs are what push the decision away from ChromaDB and toward something like pgvector.

The main options

Option	Type	Where it fits
pgvector	Extension for Postgres	Keeps vectors next to your normal data, so one system and one query can handle both. Best when you already run Postgres and have a moderate amount of data.
Qdrant / Weaviate	Dedicated, open source	Stronger filtering and hybrid search features, and they scale further. Need their own service to run and monitor.
Pinecone	Fully managed	Very little operations work and quick to start. Usage-based pricing and less control over internals.

Option	Type	Where it fits
Milvus	Distributed, open source	Built for very large scale, into the billions of vectors. Heavier to operate.
Chroma / FAISS	Embedded library	Great for prototypes, notebooks and small apps (I used ChromaDB for this reason). Not meant for large multi-user production systems.

A hypothetical problem

Imagine an internal knowledge assistant for a company of about 3,000 employees. It searches around 3 million chunks taken from wikis, shared drives, and support tickets. These requirements matter most:

- Employees must only get answers from documents they have permission to read.
- People search for exact ticket IDs and product names as well as for concepts, so hybrid search is needed.
- Traffic is moderate, tens of queries per second at peak, not thousands.
- The platform team is small, and the company already runs PostgreSQL.

My choice: PostgreSQL with pgvector

Permissions stay simple. Access rules are ordinary relational data. With pgvector, checking them is just a WHERE clause in the same query as the similarity search. With a separate vector database, I would have to copy permissions into it and keep them in sync, and mistakes there become security leaks.

The scale fits. A few million vectors is comfortable for pgvector with an HNSW index. A distributed system would add complexity I would not need.

Hybrid search is easy. Postgres already has full-text search, so keyword and vector scores can be combined in one query.

Less to operate. The team already knows how to back up, secure, and monitor Postgres. Deleting a document or changing its permissions also updates its chunks in the same transaction, which matters for compliance.

Trade-offs I would accept

One thing I learned from reading the pgvector documentation is that with approximate indexes, the filter is applied after the index search. If a filter only matches about 10% of rows and the default search width is used, a query may return only around 4 matching results out of the 40 it looked at. Version 0.8.0 added iterative index scans, which keep searching until enough results pass the filter, so this is much better than before, but it is something I would test carefully with real permission filters. HNSW indexes also use a lot of RAM. And if the data grew to tens of millions of vectors or traffic grew a lot, I would move to Qdrant or Milvus.

When I would choose differently

If there were no platform team and we needed to launch quickly, I would pick a managed service like Pinecone. If we needed heavy filtering at a much larger scale, Qdrant or Weaviate. For a prototype or a notebook, Chroma or FAISS, which is what I used for my own project.

To be transparent, my hands-on experience is with ChromaDB, in my RAG assistant. My knowledge of pgvector comes from its documentation, so in practice I would build a small prototype with real data and permission filters before committing to it.